HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA ĐÀO TẠO SAU ĐẠI HỌC

BÁO CÁO BÀI TẬP LỚN

MÔN: TÌM KIẾM VÀ TRUY XUẤT THÔNG TIN

Đề tài: No More K-means: Single-Stage Sparse Coding for Efficient Multi-Vector Retrieval

Giảng viên hướng dẫn : **TS. Đào Thị Thúy Quỳnh**
Lớp : **M25CQHT01-B**
Học viên nhóm X : **Nguyễn Văn A - B25CHHT000**
: **Nguyễn Văn B - B25CHHT000**
: **Nguyễn Văn C - B25CHHT000**
: **Nguyễn Văn D - B25CHHT000**

Hà Nội - 2026

PHÂN CÔNG NHIỆM VỤ NHÓM THỰC HIỆN

TT	Công việc / Nhiệm vụ	Học viên thực hiện
1	Điều phối, tổng hợp; viết báo cáo, làm slide	Nguyễn Văn A - B25CHHT000
2	Nghiên cứu lý thuyết (multi-vector retrieval, sparse autoencoder, SSR)	Nguyễn Văn B - B25CHHT000
3	Cài đặt môi trường, chạy lại mã nguồn, rà soát và sửa lỗi theo bài báo	Nguyễn Văn C - B25CHHT000
4	Tối ưu thuật toán truy hồi, đo và so sánh với ColBERTv2/PLAID	Nguyễn Văn D - B25CHHT000

NHÓM THỰC HIỆN TỰ ĐÁNH GIÁ

TT	Học viên	Thái độ tham gia	Mức hoàn thành CV	Kỹ năng giao tiếp	Kỹ năng hợp tác	Kỹ năng lãnh đạo
1	Nguyễn Văn A - B25CHHT000	5	5	5	5	5
2	Nguyễn Văn B - B25CHHT000	5	5	5	5	4
3	Nguyễn Văn C - B25CHHT000	5	5	5	5	4
4	Nguyễn Văn D - B25CHHT000	5	5	5	5	4

Ghi chú: thang điểm 0–5 cho từng tiêu chí (0: không tham gia / không hoàn thành; 5: chủ động, tích cực / hoàn thành xuất sắc).

MỤC LỤC

MỞ ĐẦU **1**

1 Tổng quan bài báo **3**
1.1 Thông tin xuất bản 3
1.2 Tóm tắt nội dung 3
1.3 Đóng góp chính của bài báo 3
1.4 Vị trí của bài báo trong lĩnh vực 4

2 Cơ sở lý thuyết và nghiên cứu liên quan **5**
2.1 Bài toán truy hồi và chỉ mục ngược 5
2.2 Truy hồi đa vector và phép tương tác muộn MaxSim 5
2.2.1 Pipeline ba giai đoạn của ColBERTv2/PLAID 5
2.3 Bộ tự mã hóa thưa TopK 6
2.4 Thuật toán SSR 6
2.4.1 Chấm điểm MaxSim thưa 6
2.4.2 Huấn luyện lại 6
2.4.3 Chỉ mục ngược mức neuron 7
2.4.4 Tăng tốc SSR++ 7
2.5 Các nghiên cứu liên quan 7
2.6 Độ đo đánh giá và bộ dữ liệu 7

3 Triển khai thuật toán **9**
3.1 Dữ liệu và môi trường thực nghiệm 9
3.1.1 Dữ liệu 9
3.1.2 Môi trường 9
3.1.3 Cấu trúc mã nguồn công bố 9
3.2 Kiểm tra pipeline đánh giá bằng baseline 10
3.3 Rà soát mã nguồn: các lỗi phát hiện và cách sửa 10
3.4 Kết quả chạy lại trên split test 11
3.5 Kiểm tra theo split: điểm chỉ vọt lên ở split test 12
3.6 Đánh giá in-domain trên MS MARCO 13
3.7 So sánh hiệu năng với ColBERTv2 + PLAID 13
3.8 Tối ưu thuật toán truy hồi 14
3.8.1 Vị trí tối ưu trong pipeline truy hồi 14
3.8.2 Kernel exact mới `fast_hyb` 15
3.8.3 Kernel exact trên GPU 15
3.8.4 Phương án xấp xỉ mới: LB-rerank 16
3.8.5 Các hướng đã thử nhưng không hiệu quả 16
3.9 Thông tin từ các issue và các phiên bản của bài báo 17
3.9.1 Trả lời của tác giả trên GitHub 17
3.9.2 Các phiên bản của bài báo 17

3.10 Giới hạn của thực nghiệm . . . 17

4 Kết luận . . . 18
4.1 Điểm mạnh của phương pháp SSR . . . 18
4.2 Hạn chế của bài báo và mô hình . . . 18
4.3 Khả năng ứng dụng thực tiễn . . . 18
4.4 Hướng phát triển tiếp theo . . . 19

KẾT LUẬN . . . 20

TÀI LIỆU THAM KHẢO . . . 21

A Chi tiết các lỗi trong mã nguồn và bản sửa . . . 22
A.1 Lỗi 2: nạp checkpoint tạo SAE ngẫu nhiên . . . 22
A.2 Lỗi 1: công thức chấm điểm trên chỉ mục ngược . . . 22
A.3 Lỗi 3 và 4: ánh xạ neuron và bước thô SSR++ . . . 23
A.4 Lỗi 5–7 và 9: độ đo và huấn luyện . . . 23

DANH MỤC CÁC HÌNH ẢNH

2.1 So sánh pipeline PLAID (ba giai đoạn, cần K-means) và SSR (một giai đoạn trên chỉ mục ngược), vẽ lại theo Hình 2 của bài báo. 5

3.1 Chất lượng trên split sạch và split test. Chỉ SSR-tok (có SAE) có bước nhảy lớn ở split test. 12
3.2 Pipeline truy hồi của SSR và vị trí các tối ưu của nhóm. Hàng trên: dựng chỉ mục; hàng giữa: truy hồi exact (công thức (4)); hàng dưới: đường xấp xỉ, trong đó bước thô của SSR++ được thay bằng LB-rerank. 14
3.3 Độ trễ truy vấn trên FiQA. Mọi phương án của SSR trừ SSR++ đều cho kết quả xếp hạng giống hệt bản gốc. 16

DANH MỤC CÁC BẢNG

3.1 Các tập dữ liệu đã dùng. 9
3.2 Tái lập số đã công bố của các baseline bằng pipeline của nhóm. 10
3.3 Các sai lệch giữa mã nguồn công bố và bài báo. 11
3.4 nDCG@10 trên split test. “Repo gốc” = checkpoint công bố + mã gốc (đã sửa lỗi nạp SAE); “đã sửa” = MaxSim đúng công thức (4); SSR++ theo cấu hình Phụ lục D.1. 11
3.5 nDCG@10 trên split sạch (train/dev) và split test của cùng một tập. Quora dùng quy ước BEIR bỏ tài liệu trùng id với query. 12
3.6 Re-ranking BM25 top-100 trên MS MARCO. Pipeline SSR được kiểm tra lại: mã thưa của query trùng 235/235 dòng với hàm của repo, điểm khớp kernel exact tới $1{,}4 \cdot 10^{-6}$. 13
3.7 Hiệu năng trên FiQA. Thời gian mã hóa corpus không tính vào “dựng chỉ mục” (ColBERTv2 ≈260 s, SSR ≈420 s trên GPU). 13
3.8 Đối chiếu từng tuyên bố về hiệu năng của bài báo với số đo. 14
3.9 Từng bước của pipeline: cách làm của bản gốc (repo đã sửa) và của bản tối ưu. 15
3.10 Độ trễ của các kernel exact (kết quả xếp hạng giống hệt bản gốc). 15
3.11 Các phương án xấp xỉ trên FiQA dev (6 luồng; exact = 82 ms, nDCG@10 22,80). 16
3.12 Thông tin từ trả lời của tác giả và đối chiếu của nhóm. 17

DANH MỤC CÁC TỪ VIẾT TẮT

Viết tắt	**Cụm từ tiếng Anh**	**Giải thích**
IR	Information Retrieval	Truy xuất thông tin
MVR	Multi-Vector Retrieval	Truy hồi đa vector (mỗi tài liệu là nhiều vector token)
SVR	Single-Vector Retrieval	Truy hồi đơn vector
MaxSim	Maximum Similarity	Phép tương tác muộn của ColBERT: tổng theo token query của độ tương đồng lớn nhất với các token tài liệu
SAE	Sparse Autoencoder	Bộ tự mã hóa thưa
TopK	Top-K activation	Chỉ giữ K giá trị lớn nhất, còn lại đặt bằng 0
SSR	Single-stage Sparse Retrieval	Truy hồi thưa một giai đoạn (phương pháp của bài báo)
SSR++	SSR with pruning	SSR có thêm bước lọc thô rồi chấm tinh
PLAID	Performance-optimized Late Interaction Driver	Bộ máy truy hồi của ColBERTv2 dùng K-means và nén residual
BM25	Best Matching 25	Hàm xếp hạng từ vựng cổ điển
BEIR	Benchmarking IR	Bộ 18 tập dữ liệu đánh giá truy hồi zero-shot
nDCG	normalized Discounted Cumulative Gain	Độ lợi tích lũy chiết khấu chuẩn hóa
MRR	Mean Reciprocal Rank	Trung bình nghịch đảo thứ hạng
VRAM	Video RAM	Bộ nhớ của GPU

MỞ ĐẦU

1. Lý do chọn đề tài

Truy hồi thần kinh (neural retrieval) luôn phải cân bằng giữa độ chính xác và chi phí. Các mô hình đơn vector (SVR) nén cả văn bản thành một vector nên tìm kiếm nhanh, nhưng mất chi tiết ngữ nghĩa. Các mô hình đa vector (MVR) như ColBERT giữ một vector cho mỗi token và chấm điểm bằng phép tương tác muộn MaxSim, cho chất lượng cao hơn rõ rệt nhưng chỉ mục lớn gấp hàng chục lần. Để triển khai được, các hệ thống như ColBERTv2/PLAID phải nén vector và phân cụm hàng tỷ token bằng K-means, khiến việc dựng chỉ mục tốn nhiều giờ và quy trình truy hồi gồm nhiều giai đoạn: lọc, giải nén rồi chấm lại.

Bài báo *No More K-means: Single-Stage Sparse Coding for Efficient Multi-Vector Retrieval* (Guo và cộng sự, ICML 2026) [1] đề xuất một hướng khác: thay vì nén vector token xuống chiều thấp, dùng một bộ tự mã hóa thưa (SAE) chiếu chúng lên không gian 16.384 chiều nhưng chỉ có 32 chiều khác 0. Nhờ đó, mỗi chiều hoạt động đóng vai trò một "từ giả" và có thể dùng thẳng chỉ mục ngược của tìm kiếm từ khóa, không cần K-means. Bài báo tuyên bố giảm 15 lần thời gian dựng chỉ mục, giảm một nửa độ trễ truy vấn và đồng thời tăng chất lượng so với các baseline mạnh.

Đề tài phù hợp với môn học vì liên quan trực tiếp đến hai nội dung chính của môn: chỉ mục ngược trong truy hồi từ vựng và các mô hình truy hồi thần kinh. Bài báo mới công bố, có mã nguồn và checkpoint, nhưng trên GitHub đã có hai nhóm độc lập báo cáo không tái lập được kết quả. Việc tự chạy lại, rà soát mã nguồn và kiểm chứng từng tuyên bố giúp nhóm rèn luyện cả kỹ năng cài đặt lẫn phương pháp đánh giá.

2. Mục tiêu của báo cáo

- Trình bày cơ sở lý thuyết của truy hồi đa vector, PLAID, bộ tự mã hóa thưa TopK và thuật toán SSR/SSR++ của bài báo.
- Chạy lại mã nguồn và checkpoint do tác giả công bố; đối chiếu mã nguồn với các công thức trong bài, phát hiện và sửa những chỗ sai lệch; kiểm chứng kết quả trên các tập dữ liệu chuẩn BEIR và MS MARCO.
- Tối ưu thuật toán truy hồi chính (chấm điểm MaxSim thưa trên chỉ mục ngược) và so sánh với bản gốc cũng như với ColBERTv2 + PLAID về tốc độ, bộ nhớ và chất lượng trên cùng một máy.
- Đánh giá các tuyên bố của bài báo dựa trên số liệu đo được và rút ra kinh nghiệm về tái lập nghiên cứu.

3. Đối tượng và phạm vi nghiên cứu

a. Đối tượng nghiên cứu: phương pháp SSR (Single-stage Sparse Retrieval) gồm bộ tự mã hóa thưa TopK trên nền BERT-base, phép chấm điểm MaxSim thưa (công thức (4) của bài báo) trên chỉ mục ngược mức neuron, và chiến lược tăng tốc SSR++.

b. Phạm vi nghiên cứu:

- *Lý thuyết:* bài báo arXiv:2605.30120 (đối chiếu các bản v3, v4 và bản chính thức trên PMLR 306).
- *Thực hành:* mã nguồn chính thức https://github.com/Y-Research-SBU/SSR (commit `c64bc05`) và checkpoint `SSR-tok` trên HuggingFace; các tập dữ liệu NFCorpus, SciFact, FiQA-2018, Quora (BEIR) và MS MARCO passage; baseline đối chứng ColBERTv2 (PLAID). Phần huấn luyện lại từ đầu nằm ngoài phạm vi vì vượt quá tài nguyên phần cứng của nhóm (một GPU 8 GB).

4. Phương pháp thực hiện

- **Tổng hợp và phân tích tài liệu:** đọc bài báo gốc và các công trình nền (ColBERT, ColBERTv2, PLAID, SPLADE, TopK SAE), tính lại các bảng số liệu trong bài và so sánh các phiên bản của bài.
- **Phân tích mã nguồn:** đọc toàn bộ repository, đối chiếu từng hàm với công thức tương ứng, đọc các issue trên GitHub và trả lời của tác giả.
- **Thực nghiệm có đối chứng:** mọi phép đo đều chạy trên cùng một máy, dùng cùng pipeline cho SSR và ColBERTv2; pipeline được kiểm tra bằng cách tái lập số đã công bố của ColBERTv2 và BM25 trước khi đo SSR. Kết quả của mỗi bản sửa được kiểm chứng bằng ít nhất hai cách tính độc lập.
- **Tối ưu và đo lường:** thiết kế lại kernel chấm điểm, viết test so sánh từng bit với bản gốc, đo độ trễ p50/p95 và bộ nhớ.

5. Cấu trúc báo cáo

Ngoài phần Mở đầu, Kết luận và Tài liệu tham khảo, báo cáo gồm 4 chương:

- **Chương 1. Tổng quan bài báo:** thông tin xuất bản, tóm tắt nội dung, đóng góp và vị trí của bài báo.
- **Chương 2. Cơ sở lý thuyết và nghiên cứu liên quan:** truy hồi đa vector và MaxSim, pipeline PLAID, bộ tự mã hóa thưa TopK, thuật toán SSR/SSR++ và các độ đo đánh giá.
- **Chương 3. Triển khai thuật toán:** dữ liệu, môi trường, cấu trúc mã nguồn, các lỗi đã phát hiện và cách sửa, kết quả chạy lại, kiểm tra theo split, kết quả MS MARCO, so sánh hiệu năng với PLAID và phần tối ưu thuật toán.
- **Chương 4. Kết luận:** điểm mạnh, hạn chế của bài báo và mô hình, khả năng ứng dụng và hướng phát triển.

CHƯƠNG 1. TỔNG QUAN BÀI BÁO

1.1. Thông tin xuất bản

- **Tên bài báo:** No More K-means: Single-Stage Sparse Coding for Efficient Multi-Vector Retrieval.
- **Tác giả:** Lixuan Guo*, Yifei Wang*, Tiansheng Wen, Aosong Feng, Stefanie Jegelka, Chenyu You (* đóng góp ngang nhau).
- **Đơn vị:** Stony Brook University, Xidian University, Amazon AGI SF Lab, Georgia Tech, Yale University, TUM, MIT.
- **Nơi công bố:** Proceedings of the 43rd International Conference on Machine Learning (ICML 2026), PMLR 306. Bản tiền ấn phẩm arXiv:2605.30120 (v3 ngày 03/06/2026, v4 ngày 28/09/2026).
- **Mã nguồn:** https://github.com/Y-Research-SBU/SSR (giấy phép MIT). Checkpoint: https://huggingface.co/Y-Research-Group/SSR.
- **Lĩnh vực:** Information Retrieval, Multi-Vector Retrieval, Sparse Representation Learning.

1.2. Tóm tắt nội dung

Bài báo xuất phát từ một quan sát về các hệ thống truy hồi đa vector hiện nay: để lưu và tìm kiếm trên hàng tỷ vector token, ColBERTv2/PLAID phải (i) giảm chiều và lượng tử hóa vector, (ii) phân cụm toàn bộ token bằng K-means để tạo codebook, rồi (iii) truy hồi theo ba giai đoạn: sinh ứng viên từ centroid, chấm điểm xấp xỉ để cắt tỉa, cuối cùng giải nén và chấm MaxSim chính xác. Quy trình này tốn thời gian dựng chỉ mục (K-means trên hàng tỷ điểm) và làm mất thông tin do nén.

Thay vào đó, SSR chiếu mỗi vector token $x \in \mathbb{R}^{768}$ của BERT qua một bộ tự mã hóa thưa TopK thành $z \in \mathbb{R}^{16384}$ chỉ có $K = 32$ thành phần khác 0. Điểm liên quan giữa query và tài liệu vẫn là MaxSim, nhưng tích vô hướng giữa hai token chỉ tính trên các neuron cùng kích hoạt. Vì mỗi neuron hoạt động giống một từ trong từ điển, tài liệu được đưa vào chỉ mục ngược theo neuron, giống hệt BM25; truy hồi chỉ cần duyệt $|Q| \times K$ danh sách posting. Bài còn đề xuất SSR++: lọc thô bằng $K_{\text{coarse}} = 4$ neuron lớn nhất của mỗi token query trên các posting lưu giá trị lớn nhất theo tài liệu, rồi chấm chính xác trên 2.000 ứng viên.

Mô hình được huấn luyện bằng mục tiêu lai: tái tạo (reconstruction) của SAE, số hạng phụ AuxK chống "neuron chết", một số hạng đối lập (contrastive) và tổn thất xếp hạng trên điểm MaxSim. Thực nghiệm trên MS MARCO và 13 tập BEIR cho thấy SSR-tok đạt trung bình 52,9 nDCG@10 (ColBERTv2: 49,2; SPLADE-v3: 51,2), độ trễ 17,5 ms/query (PLAID 37,1 ms) và thời gian dựng chỉ mục 7,3 giờ so với 122,9 giờ của ColBERTv2 trên MS MARCO.

1.3. Đóng góp chính của bài báo

- **Mô hình truy hồi thưa một giai đoạn:** thay K-means và nén residual bằng sparse coding, cho phép dùng trực tiếp chỉ mục ngược cho truy hồi ngữ nghĩa đa vector.
- **Mục tiêu huấn luyện lai:** kết hợp tổn thất tái tạo của SAE với tổn thất đối lập đa vector để các đặc trưng thưa vừa tái tạo tốt vừa có tính phân biệt cho xếp hạng.

- **Đánh giá rộng:** in-domain (MS MARCO) và out-of-domain (BEIR), mở rộng sang backbone LLM 8B (Llama-Embed-Nemotron-8B), kèm phân tích lý thuyết về sai số có chặn của phép chấm điểm thưa (Phụ lục A).

1.4. Vị trí của bài báo trong lĩnh vực

Bài báo nằm ở giao điểm của hai dòng nghiên cứu. Dòng thứ nhất là truy hồi đa vector hiệu quả: ColBERT (2020) → ColBERTv2 và PLAID (2022) → EMVB, XTR, CITADEL (2023–2024) đều tìm cách giảm chi phí MaxSim bằng nén, phân cụm hoặc cắt tỉa token. Dòng thứ hai là truy hồi thưa học được (learned sparse retrieval) như SPLADE, dùng từ điển của BERT làm không gian thưa để tận dụng chỉ mục ngược. SSR kết hợp hai hướng này: giữ tương tác giữa các token của ColBERT nhưng biểu diễn thưa học được từ SAE, không ràng buộc vào từ điển. Đồng thời, bài là một trong những ứng dụng đầu tiên của TopK SAE (vốn phát triển cho diễn giải mô hình ngôn ngữ) vào bài toán truy hồi.

Ở thời điểm thực hiện báo cáo (10/2026), mã nguồn của bài có hai issue công khai từ hai nhóm độc lập báo không tái lập được Bảng 1, tác giả đã thừa nhận một số lỗi và bản arXiv v4 đã sửa lại cách so sánh trên MS MARCO. Vì vậy, bên cạnh việc tìm hiểu ý tưởng, kiểm tra khả năng tái lập của bài báo là nội dung chính của Chương 3.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN

2.1. Bài toán truy hồi và chỉ mục ngược

Cho kho tài liệu $\mathcal{D} = \{D_1, \ldots, D_{|\mathcal{D}|}\}$ và truy vấn Q, hệ thống truy hồi cần học hàm điểm $s_\theta(Q, D)$ sao cho tài liệu liên quan D^+ có điểm cao hơn tài liệu không liên quan D^-, rồi trả về danh sách top-k theo điểm. Với truy hồi từ vựng (BM25 [10]), hàm điểm là tổng đóng góp của các từ chung giữa Q và D, nên tài liệu được tổ chức thành *chỉ mục ngược*: mỗi từ t có một danh sách posting $I_t = \{(D, w_{D,t})\}$ gồm các tài liệu chứa t và trọng số tương ứng. Truy hồi chỉ cần duyệt $|Q|$ danh sách và cộng dồn điểm, chi phí tỉ lệ với tổng độ dài các posting chứ không với kích thước kho. Chỉ mục ngược còn hỗ trợ cắt tỉa an toàn (WAND, block-max WAND): mỗi khối posting lưu cận trên của trọng số để bỏ qua những khối không thể lọt vào top-k.

2.2. Truy hồi đa vector và phép tương tác muộn MaxSim

ColBERT [2] mã hóa query thành $\{q_1, \ldots, q_N\}$ và tài liệu thành $\{d_1, \ldots, d_M\}$ (mỗi token một vector 128 chiều đã chuẩn hóa) rồi tính

$$S(Q, D) = \sum_{i=1}^{N} \max_{j=1}^{M} q_i \cdot d_j. \tag{2.1}$$

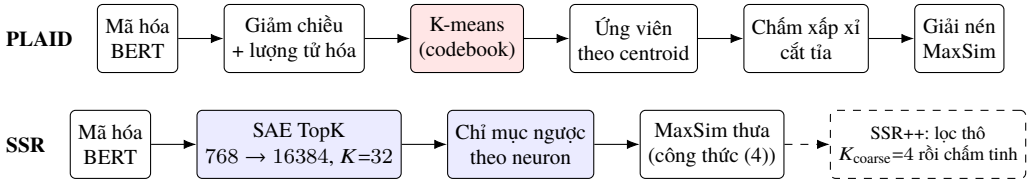
Mỗi token query "chọn" token tài liệu giống nó nhất, nên mô hình bắt được sự khớp ngữ nghĩa ở mức từ mà SVR bỏ qua. Đổi lại, chỉ mục chứa M vector cho mỗi tài liệu (MS MARCO: khoảng 600 triệu token), và chấm điểm có độ phức tạp $O(N \times M)$ cho mỗi cặp.

2.2.1. Pipeline ba giai đoạn của ColBERTv2/PLAID

ColBERTv2 [3] nén mỗi vector token thành chỉ số centroid gần nhất (K-means với $\sqrt{16 \cdot \text{số token}}$ cụm) cộng với residual lượng tử hóa 1–2 bit. PLAID [4] truy hồi theo ba giai đoạn (Hình 2.1, hàng trên):

1. **Sinh ứng viên:** với mỗi token query tìm vài centroid gần nhất; ứng viên là hợp của các tài liệu có token thuộc các centroid đó: $\mathcal{D}_{\text{cand}} = \bigcup_{q_i \in Q} \bigcup_{c \in C_i} \phi(c)$ (công thức (1) của bài).
2. **Chấm xấp xỉ và cắt tỉa:** dùng tích vô hướng giữa token query và centroid thay cho vector thật, $\hat{S}(Q, D) = \sum_i \max_j q_i \cdot c_{d_j}$ (công thức (2)), giữ lại vài nghìn tài liệu.
3. **Giải nén và xếp hạng lại:** tái tạo $\tilde{d}_j \approx c_{d_j} + r_{d_j}$ và chấm MaxSim đầy đủ (công thức (3)).

Bài báo chỉ ra hai hạn chế: dựng chỉ mục chậm vì K-means trên hàng tỷ token, và pipeline nhiều giai đoạn có logic điều khiển và truy cập bộ nhớ phức tạp.



Hình 2.1: So sánh pipeline PLAID (ba giai đoạn, cần K-means) và SSR (một giai đoạn trên chỉ mục ngược), vẽ lại theo Hình 2 của bài báo.

2.3. Bộ tự mã hóa thưa TopK

Bộ tự mã hóa thưa (SAE) học biểu diễn $z \in \mathbb{R}^h$ với $h \gg d$ sao cho x tái tạo được từ ít thành phần khác 0. Biến thể TopK [7] ép độ thưa trực tiếp:

$$z = \mathrm{TopK}\big(W_{\mathrm{enc}}(x - b_{\mathrm{pre}}) + b_{\mathrm{enc}}\big), \tag{2.2}$$

$$\hat{x} = W_{\mathrm{dec}}\, z + b_{\mathrm{pre}}, \tag{2.3}$$

trong đó $W_{\mathrm{enc}} \in \mathbb{R}^{h\times d}$, $W_{\mathrm{dec}} \in \mathbb{R}^{d\times h}$, phép TopK giữ K giá trị lớn nhất (sau ReLU) và đặt phần còn lại bằng 0. Tổn thất tái tạo là $\mathcal{L}_{\mathrm{recon}}(k) = \|x - \hat{x}\|_2^2$. Hai kỹ thuật kèm theo: (i) *AuxK* dùng k_{aux} neuron "chết" (lâu không kích hoạt) để tái tạo phần dư, giữ cho từ điển neuron không sụp về vài trăm chiều; (ii) *annealing* K từ $2K$ về K trong giai đoạn đầu huấn luyện. Trong SSR, $d = 768$ (BERT-base), $h = 16384$, $K = 32$, $k_{\mathrm{aux}} = 2048$.

2.4. Thuật toán SSR

2.4.1. Chấm điểm MaxSim thưa

Sau SAE, query thành $Q' = \{z_{q_1}, \ldots, z_{q_N}\}$ và tài liệu thành $D' = \{z_{d_1}, \ldots, z_{d_M}\}$, mỗi vector chỉ có K thành phần khác 0. Gọi $A_K(z)$ là tập chỉ số của K neuron hoạt động. Điểm liên quan là MaxSim trong không gian thưa, tích vô hướng chỉ tính trên các neuron chung:

$$S(Q, D) = \sum_{i=1}^{N} \max_{j=1}^{M} \left(\sum_{u \in A_K(z_{q_i}) \cap A_K(z_{d_j})} z_{q_i}^{(u)}\, z_{d_j}^{(u)} \right). \tag{2.4}$$

Phụ lục A của bài chứng minh rằng nếu sai số tái tạo nhỏ và các cột của W_{dec} gần trực giao trên các support hoạt động thì $z_{q_i}^{\top} z_{d_j} \approx q_i^{\top} d_j$ với sai số có chặn, và chặn này truyền qua phép max/sum của MaxSim.

Thứ tự các phép toán trong (2.4) quan trọng: với mỗi cặp (token query i, token tài liệu j) phải *cộng* các tích trên neuron chung trước, rồi mới lấy *max* theo j, cuối cùng cộng theo i. Mã nguồn công bố làm sai đúng ở thứ tự này (mục A.2).

2.4.2. Huấn luyện lai

Bài huấn luyện hai SAE: E_{tok} cho token thường (SSR-tok) và $E_{[\mathrm{CLS}]}$ cho token toàn cục (SSR-CLS). Mục tiêu không giám sát là

$$\mathcal{L}_{\mathrm{unsup}} = \mathcal{L}_{\mathrm{recon}}(k) + \tfrac{1}{8}\mathcal{L}_{\mathrm{recon}}(4k) + \alpha\, \mathcal{L}_{\mathrm{aux}}(k_{\mathrm{aux}}) + \beta\, \mathcal{L}_{\mathrm{cl}}, \tag{2.5}$$

trong đó số hạng $\frac{1}{8}\mathcal{L}_{\mathrm{recon}}(4k)$ (tái tạo với $4K$ neuron) giúp các neuron ngoài top-K vẫn được học, $\mathcal{L}_{\mathrm{cl}}$ là tổn thất đối lập trên biểu diễn thưa. Mục tiêu giám sát là tổn thất xếp hạng cross-entropy trên điểm MaxSim thưa giữa query, tài liệu dương và các hard negative của BM25:

$$\mathcal{L} = \mathcal{L}_{\mathrm{unsup}} + \gamma\, \mathcal{L}_{\mathrm{CE}}, \qquad \mathcal{L}_{\mathrm{CE}} = -\log \frac{\exp(S(Q, D^+)/\tau)}{\sum_{D \in \{D^+\} \cup \mathcal{N}} \exp(S(Q, D)/\tau)}. \tag{2.6}$$

Bảng 6 của bài cho $(\alpha, \beta, \gamma) = (0{,}03125;\ 0{,}1;\ 0{,}05)$, AdamW, lr 10^{-3}, batch 64, 2 epoch, warmup 20.000 bước trên MS MARCO. Nhiệt độ τ và số hard negative không ghi trong bài; theo trả lời của tác giả trên GitHub, $\tau = 0{,}3$, 32 hard negative mỗi query được gom vào cùng batch, BERT không đóng băng và lr thực dùng khoảng 10^{-6}.

2.4.3. Chỉ mục ngược mức neuron

Với mỗi neuron $u \in \{1, \ldots, h\}$, bài lưu giá trị lớn nhất của u trong tài liệu D:

$$\mu_{D,u} = \max_{t \in D} z_t^{(u)}, \tag{2.7}$$

và danh sách posting $I_u = \{(D, \mu_{D,u}) \mid \mu_{D,u} > 0\}$, chia thành khối cố định với cận trên $UB = \max_{D \in B} \mu_{D,u}$ để cắt tỉa sớm. Công thức (2.7) chỉ là bản tóm tắt theo tài liệu; để tính đúng (2.4) vẫn cần giá trị theo từng token, tức posting phải mang bộ ba (tài liệu, token, giá trị). Mã nguồn công bố có lưu posting mức token, nhưng phần chấm điểm lại dùng chúng như posting mức tài liệu (mục A.2).

2.4.4. Tăng tốc SSR++

SSR++ gồm hai bước, lọc thô rồi chấm tinh:

1. **Lọc thô:** chỉ dùng $K_{\text{coarse}} = 4$ neuron lớn nhất của mỗi token query và posting μ:

$$\hat{S}_{\text{coarse}}(Q, D) = \sum_{i=1}^{N} \sum_{u \in A_{K_{\text{coarse}}}(q_i)} q_i^{(u)} \mu_{D,u}, \tag{2.8}$$

   kết hợp bỏ qua các khối theo UB, giữ lại tập ứng viên C_1 (2.000 tài liệu theo Phụ lục D.1).

2. **Chấm tinh:** tính (2.4) với đủ K neuron chỉ trên C_1.

Độ phức tạp của SSR là $O(N \cdot K \cdot \bar{L})$ với $\bar{L}$ là độ dài posting trung bình; SSR++ giảm bước 1 còn $O(N \cdot K_{\text{coarse}} \cdot L_{\text{skip}})$. Công thức (2.8) không phải cận trên của (2.4) (nó bỏ $K - K_{\text{coarse}}$ neuron) nên SSR++ là phép xấp xỉ có thể mất tài liệu liên quan; bài báo cho rằng mức mất là không đáng kể (Bảng 5).

2.5. Các nghiên cứu liên quan

- **SPLADE** [6]: học trọng số thưa trên từ điển 30.522 từ của BERT, truy hồi bằng chỉ mục ngược. Khác SSR ở chỗ SPLADE gộp cả văn bản thành một vector thưa (đơn vector) và không gian thưa là từ điển, còn SSR giữ đa vector và không gian thưa học được.
- **COIL, CITADEL, XTR:** các biến thể ColBERT dùng từ vựng hoặc bộ định tuyến để giới hạn token nào tương tác với token nào; SSR đi xa hơn khi coi mỗi neuron SAE là một "từ".
- **EMVB** [5]: tăng tốc PLAID bằng bit-vector và SIMD; là đối thủ trực tiếp về độ trễ, được so sánh trong Bảng 15 của bài.
- **TopK SAE** [7]: nguồn gốc kiến trúc SAE của bài; khác biệt là SSR huấn luyện SAE cùng tín hiệu xếp hạng thay vì chỉ tái tạo.

2.6. Độ đo đánh giá và bộ dữ liệu

nDCG@10 [11]: với danh sách xếp hạng $r_1, \ldots, r_{10}$ có mức liên quan rel_k, $\text{DCG@10} = \sum_{k=1}^{10} \frac{2^{\text{rel}_k} - 1}{\log_2(k+1)}$ và $\text{nDCG@10} = \text{DCG@10}/\text{IDCG@10}$, trong đó IDCG là DCG của xếp hạng lý tưởng. BEIR dùng mức liên quan có trọng số (NFCorpus có 3 mức, TREC-DL có 4 mức). **MRR@10** là trung bình của $1/\text{rank}$ của tài liệu liên quan đầu tiên trong top-10 (bằng 0 nếu không có), dùng cho MS MARCO dev. Hai độ đo này *không* so sánh được với nhau: trên cùng một hệ thống, nDCG@10 thường cao hơn MRR@10 vài điểm.

BEIR [8] gồm 18 tập dữ liệu truy hồi zero-shot thuộc nhiều lĩnh vực; bài dùng 13 tập. Báo cáo này dùng bốn tập có đủ split: NFCorpus (y sinh, 3.633 tài liệu; split train/dev/test), SciFact (xác minh khẳng định khoa học,

5.183 tài liệu; train/test), FiQA-2018 (tài chính, 57.638 tài liệu; train/dev/test) và Quora (câu hỏi trùng lặp, 522.931 tài liệu; dev/test). **MS MARCO passage** [9] có 8,84 triệu đoạn văn, tập dev-small 6.980 query (nhãn nhị phân) và TREC DL 2019 với 43 query có nhãn 4 mức.

CHƯƠNG 3. TRIỂN KHAI THUẬT TOÁN

Mã nguồn của nhóm (bản repo đã sửa, kèm patch, script đánh giá và mã tối ưu): https://github.com/duykhanhxx03/ssr-repro-and-optimization.

3.1. Dữ liệu và môi trường thực nghiệm

3.1.1. Dữ liệu

Bảng 3.1: Các tập dữ liệu đã dùng.

Tập dữ liệu	Lĩnh vực	Số tài liệu	Query (test)	Split khác đã dùng
NFCorpus	y sinh	3.633	323	dev 324, train 2.590
SciFact	khoa học	5.183	300	train 809
FiQA-2018	tài chính	57.638	648	dev 500, train 1.000 (mẫu)
Quora	câu hỏi trùng	522.931	10.000	dev 5.000
MS MARCO passage	web, in-domain	8.841.823	dev-small 6.980; DL19 43	không

Các tập BEIR tải theo định dạng MTEB (`corpus.jsonl`, `queries.jsonl`, `qrels`) bằng script `prepare_mteb_eval.py` của repo, có bổ sung các split train/dev để phục vụ kiểm tra ở mục 3.5. MS MARCO tải từ bản phát hành gốc (`collection.tsv`, `queries.dev.small.tsv`, `qrels`) và qrels TREC DL 2019.

3.1.2. Môi trường

- Phần cứng: CPU Intel i5-12400F (6 nhân/12 luồng), RAM 32 GB, GPU NVIDIA RTX 5060 8 GB; Windows 11.
- Phần mềm: Python 3.13, PyTorch 2.11 (CUDA 12.8), transformers 5.3, sentence-transformers 5.3, PyLate 1.6 (ColBERT + fast-plaid), numba 0.61, numpy, scipy.
- Mã nguồn: `Y-Research-SBU/SSR` commit `c64bc05` (26/06/2026; không có commit nào mới hơn tới ngày 12/10/2026). Checkpoint `SSR-tok` tải từ HuggingFace ngày 11/10/2026 (tác giả đưa lên ngày 05/10/2026).
- Baseline: `colbert-ir/colbertv2.0` chạy qua PyLate; BM25 tự cài bằng numpy theo công thức Lucene ($k_1 = 0{,}9$, $b = 0{,}4$).

Mọi số đo thời gian đều chạy khi máy không có tác vụ khác, mỗi lần một tác vụ, sau giai đoạn khởi động (warm-up).

3.1.3. Cấu trúc mã nguồn công bố

Listing 3.1: Các thành phần chính của repository SSR.

```
SSR/
  prepare_msmarco.py         # tai va dong goi MS MARCO thanh triplet
  prepare_mteb_eval.py       # tai cac tap BEIR theo dinh dang MTEB
  ssr/
    model.py                 # lop SSR = ColBERT (PyLate) + SparseAutoencoder
    sparse_autoencoder.py    # SAE TopK, AuxK, annealing (cong thuc (5)-(6))
    losses.py                # recon, aux, contrastive, MaxSim CE (cong thuc (7)-(8))
    dataset.py, train.py     # nap triplet MS MARCO, vong lap huan luyen
    retrieval/
      inverted_index.py      # dung chi muc nguoc, cham diem tren index
      exact_retriever.py     # duong "exact" (torch.sparse / index)
```

```
    retriever.py           # SparseMaxSimRetriever (SSR++)
    eval_mteb.py           # CLI danh gia: --variant ssr|ssr-cls, --mode exact|pruned
    metrics.py, mteb_io.py # nDCG, MRR; doc qrels
  tests/
```

Lệnh đánh giá chuẩn của repo (sau khi đã sửa) là:

```
python -m ssr.retrieval.eval_mteb --task retrieval --variant ssr \
   --model-path ckpt/SSR-tok --dataset nfcorpus --data-dir data/mteb \
   --mode exact --score-device index --split test
```

3.2. Kiểm tra pipeline đánh giá bằng baseline

Trước khi đo SSR, nhóm kiểm tra rằng pipeline đánh giá (đọc dữ liệu, tính nDCG có trọng số, mã hóa, MaxSim) cho đúng số đã công bố với các mô hình đã biết. Kết quả ở Bảng 3.2 cho thấy sai lệch dưới 0,6 điểm, nằm trong biên độ khác biệt phiên bản thư viện. Nhờ đó, mọi khác biệt sau này của SSR so với bài báo không thể quy cho pipeline.

Bảng 3.2: Tái lập số đã công bố của các baseline bằng pipeline của nhóm.

Mô hình	Tập / độ đo	Đã công bố	Nhóm đo
ColBERTv2	SciFact nDCG@10	69,3	69,16
ColBERTv2	NFCorpus nDCG@10	33,8	34,40
ColBERTv2	FiQA nDCG@10	35,6	35,80
ColBERTv2	Quora nDCG@10	85,2	85,67
BM25 (Lucene)	MS MARCO dev-small MRR@10	18,4 (Anserini)	18,38
BM25 (Lucene)	TREC DL19 nDCG@10	50,6 (Anserini)	50,72

3.3. Rà soát mã nguồn: các lỗi phát hiện và cách sửa

Nhóm đối chiếu từng hàm trong repo với công thức tương ứng của bài. Bảng 3.3 tổng hợp các điểm sai lệch. Tình trạng này đã được chính tác giả công nhận trên GitHub: trong trả lời issue #1 (31/07/2026) và issue #2 (04/08/2026), tác giả xác nhận mã công bố nạp SAE ngẫu nhiên, chỉ dùng 1 hard negative thay vì 32 và thiếu số hạng $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$, đồng thời hứa đẩy bản sửa; đến ngày 12/10/2026 repo vẫn chưa có commit mới (mục 3.9). Các lỗi 1, 3, 4, 5, 8 trong bảng là phát hiện của nhóm, chưa được nêu trong các issue. Toàn bộ bản sửa được đóng gói thành một patch (16 file, áp dụng sạch lên commit `c64bc05`) kèm 7 unit test. Chi tiết từng lỗi cùng đoạn mã trước và sau khi sửa được trình bày trong Phụ lục A.

Trong các lỗi trên, lỗi 1 ảnh hưởng lớn nhất đến kết quả và chưa được nêu trong các issue trên GitHub. Bộ cộng dồn của chế độ `exact` lấy giá trị lớn nhất theo từng neuron rồi mới gộp, nên điểm của cặp (token query i, tài liệu D) trở thành $\max_u q_i^{(u)} \mu_{D,u}$ thay vì MaxSim theo (2.4): phép cộng trên các neuron chung bị mất và các neuron không bị ràng buộc phải thuộc cùng một token tài liệu. Nhóm viết lại kernel chấm điểm để cộng theo từng cặp (tài liệu, token) rồi mới lấy max theo token và cộng theo token query (Phụ lục A.2), và kiểm chứng bằng ba cách tính độc lập (MaxSim trên vector dense bằng PyTorch, một cài đặt bằng `scipy.sparse`, bản "unblocked" của repo đã sửa cùng lỗi); cả ba trùng nhau tới 10^{-6} trên mọi cặp query và tài liệu của NFCorpus và SciFact.

Các lỗi còn lại được sửa theo mô tả trong bài hoặc theo trả lời của tác giả trong issue: nạp đúng SAE và cấu hình ColBERT từ checkpoint (lỗi 2), ánh xạ chỉ số neuron cho query (lỗi 3), cài lại bước thô của SSR++ theo (2.8) với tham số của Phụ lục D.1 (lỗi 4), nDCG có trọng số (lỗi 5), thêm số hạng $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$, tham số warmup, 32 hard negative gom theo query và hai preset huấn luyện (lỗi 6, 7, 9). Đường huấn luyện đã chạy thử trọn vẹn trên một tập MS MARCO nhỏ; huấn luyện đầy đủ (khoảng 250k bước mỗi epoch với batch 64) vượt quá GPU 8 GB của nhóm.

Bảng 3.3: Các sai lệch giữa mã nguồn công bố và bài báo.

#	Lỗi	Ảnh hưởng	Theo bài	Trạng thái
1	Chế độ `exact` trên chỉ mục ngược tính $\sum_i \max_u q_i^{(u)} \max_j d_j^{(u)}$ thay vì MaxSim; đường Python, numba và hàm "tham chiếu" sai giống nhau	NFCorpus 8,33 / SciFact 7,46 → 43,97 / 87,83	Công thức (4)	Đã sửa
2	`load_ssr` dựng SAE ngẫu nhiên 3.072 latent thay vì nạp checkpoint; bỏ qua `config_sentence_transformers.json` (độ dài tài liệu 180 thay vì 512, bật query expansion)	Không nạp được mô hình	không ghi	Đã sửa
3	Đường `e2e-index` bỏ mất `latent_remap`: query và chỉ mục dùng hai hệ chỉ số neuron khác nhau	0,012 → 0,440	không ghi	Đã sửa
4	Bước thô SSR++ không theo công thức (12); tham số 8 / 32.768 / 512 thay vì $K_{\text{coarse}} = 4$, 2.000 ứng viên, khối 64	SSR++ khác bài	(11)–(12), D.1	Đã sửa
5	nDCG tính với nhãn nhị phân, bỏ mức liên quan	≈1 điểm trên NFCorpus	D.1	Đã sửa
6	Thiếu số hạng $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$ trong tổn thất	Huấn luyện	(7)	Đã sửa
7	Tham số huấn luyện mặc định lệch Bảng 6 ($\gamma = 0$ tắt $\mathcal{L}_{\text{CE}}$, lr $2 \cdot 10^{-5}$, batch 256, 1 epoch, không warmup, $k_{\text{aux}} = 1024$)	Huấn luyện	Bảng 6	Thêm preset
8	Config SAE của checkpoint có khóa lạ; dựng chỉ mục tạo một object Python cho mỗi posting (tràn RAM từ ~50k tài liệu)	Không chạy được corpus lớn	không ghi	Đã sửa
9	Bộ nạp dữ liệu chỉ lấy 1 hard negative mỗi query; tác giả dùng 32 và gom theo query (chỉ nói trong issue #1)	Huấn luyện	Không ghi	Sửa theo issue
10	`token_skip_mode` (bỏ stopword) chỉ có trong config checkpoint, mã nguồn không có	Mã hóa	Không ghi	Chưa sửa

3.4. Kết quả chạy lại trên split test

Bảng 3.4 so sánh nDCG@10 (có trọng số, chuẩn BEIR) của checkpoint công bố khi chạy bằng mã gốc (chỉ sửa lỗi nạp SAE) và bằng mã đã sửa, cùng với số trong bài và ColBERTv2 đo lại.

Bảng 3.4: nDCG@10 trên split test. "Repo gốc" = checkpoint công bố + mã gốc (đã sửa lỗi nạp SAE); "đã sửa" = MaxSim đúng công thức (4); SSR++ theo cấu hình Phụ lục D.1.

Tập (số tài liệu)	Bài báo	Repo gốc	SSR-tok đã sửa	SSR++ đã sửa	ColBERTv2
NFCorpus (3,6k)	38,8	8,3	43,97	43,98	34,40
SciFact (5,2k)	72,9	7,5	87,83	87,83	69,16
FiQA (57,6k)	39,4	không chạy	41,41	36,86	35,80
Quora (523k)	88,3	không chạy	85,42	không chạy	85,67

Listing 3.2: Trích log chạy NFCorpus bằng mã đã sửa (`logs/A_fixed_nfcorpus.log`).

```
2026-10-11 20:39:34,255 [INFO] Metrics for nfcorpus: mrr@[] ndcg@[10]
2026-10-11 20:40:02,045 [INFO]   ndcg@10: 0.4397
```

Một số nhận xét từ Bảng 3.4:

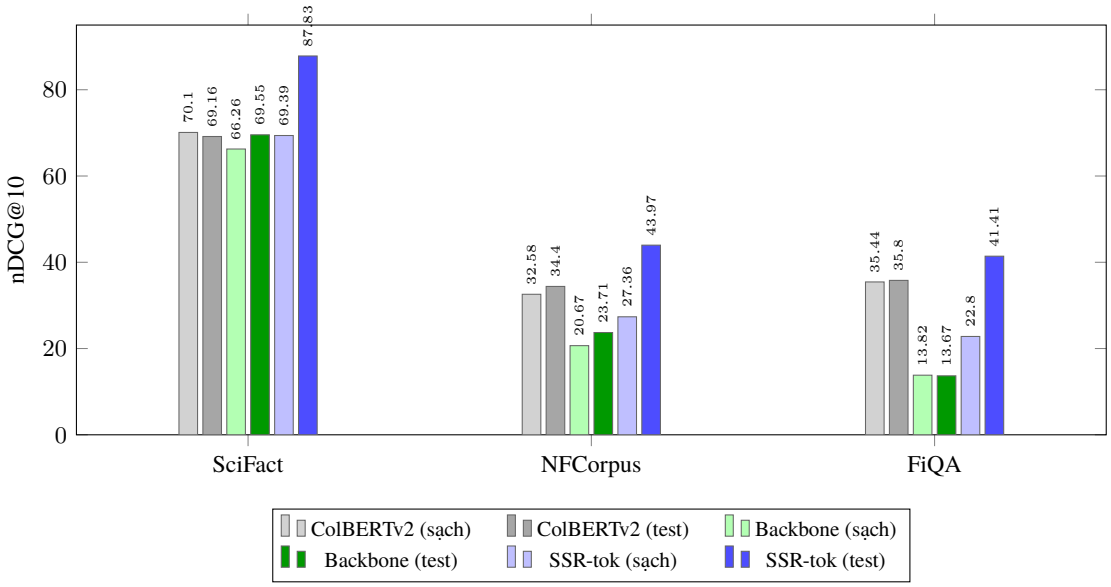
- Sau khi sửa, mã chạy đúng công thức và SSR vượt ColBERTv2 trên split test của ba tập đầu, đúng chiều với tuyên bố của bài.
- Tuy nhiên checkpoint công bố cho số *cao hơn* bài báo trên 3/4 tập (SciFact 87,8 so với 72,9), nên đây không phải mô hình đã dùng cho Bảng 1. SciFact 87,8 với một mô hình cỡ BERT-base cao hơn mọi mô hình 7–8B trong Bảng 2 của chính bài, một dấu hiệu bất thường cần kiểm tra thêm (mục 3.5).
- SSR++ không mất điểm trên NFCorpus và SciFact chỉ vì 2.000 ứng viên gần như phủ hết corpus (3,6k đến 5,2k tài liệu). Trên FiQA (57,6k) nó mất 4,6 điểm, trái với Bảng 5 của bài.

3.5. Kiểm tra theo split: điểm chỉ vọt lên ở split test

Bài báo tuyên bố đánh giá zero-shot: mô hình chỉ học MS MARCO. Khi đó chất lượng trên split train/dev và split test của *cùng một* tập BEIR phải xấp xỉ nhau, vì chúng được lấy mẫu từ cùng phân phối. Nhóm đo ba hệ thống trên cùng corpus: ColBERTv2 (đối chứng), SSR-tok (có SAE), và "backbone dense" là chính BERT của checkpoint SSR nhưng bỏ qua SAE, chấm MaxSim trên vector 768 chiều.

Bảng 3.5: nDCG@10 trên split sạch (train/dev) và split test của cùng một tập. Quora dùng quy ước BEIR bỏ tài liệu trùng id với query.

Tập (split sạch → test)	Mô hình	Split sạch	Test	Test − sạch
SciFact (train → test)	ColBERTv2	70,10	69,16	−0,9
	Backbone dense (không SAE)	66,26	69,55	+3,3
	SSR-tok	69,39	87,83	+18,4
NFCorpus (dev → test)	ColBERTv2	32,58	34,40	+1,8
	Backbone dense	20,67	23,71	+3,0
	SSR-tok	27,36	43,97	+16,6
FiQA (dev → test)	ColBERTv2	35,44	35,80	+0,4
	Backbone dense	13,82	13,67	−0,2
	SSR-tok	22,80	41,41	+18,6
Quora (dev → test)	ColBERTv2	85,25	85,67	+0,4
	SSR-tok	82,16	85,42	+3,3



Hình 3.1: Chất lượng trên split sạch và split test. Chỉ SSR-tok (có SAE) có bước nhảy lớn ở split test.

Kết quả (Bảng 3.5, Hình 3.1) cho thấy:

- ColBERTv2 phẳng giữa các split (chênh dưới 2 điểm), đúng như kỳ vọng của đánh giá zero-shot. Backbone BERT của chính checkpoint SSR cũng phẳng.
- SSR-tok vọt lên 16–19 điểm chỉ ở split test của SciFact, NFCorpus, FiQA. Các split sạch khác (NFCorpus train 2.590 query: 28,76; FiQA train 1.000 query: 20,91) cho cùng kết quả như dev. Với vài trăm đến vài nghìn query mỗi split, chênh lệch cỡ này khó có thể do ngẫu nhiên.
- Vì backbone không có hiện tượng này, phần bất thường nằm ở tầng SAE: trên split sạch SAE chỉ thêm 3–9 điểm so với backbone, nhưng thêm 18–28 điểm trên split test. Hiện tượng này phù hợp với khả năng SAE từng được huấn luyện hoặc chọn mô hình khi có mặt dữ liệu test của BEIR. Mã huấn luyện công bố chỉ đọc MS MARCO, nên nhóm chỉ ghi nhận hiện tượng, không kết luận về nguyên nhân.

- Do đó, trên dữ liệu sạch, checkpoint công bố ngang hoặc kém ColBERTv2 (SciFact −0,7; NFCorpus −5,2; FiQA −12,6; Quora −3,1).

3.6. Đánh giá in-domain trên MS MARCO

MS MARCO là dữ liệu huấn luyện của SSR. Truy hồi toàn bộ 8,84 triệu đoạn văn cần chỉ mục mức token khoảng 70–190 GB RAM, vượt khả năng của máy, nên nhóm dùng giao thức re-ranking chuẩn: BM25 lấy top-100 cho mỗi query, rồi mọi mô hình chấm lại cùng tập ứng viên bằng MaxSim chính xác. Kết quả bị giới hạn bởi recall của BM25 (66,8% query dev-small có đoạn liên quan trong top-100) nên không so trực tiếp với số full retrieval trong bài; so sánh *giữa các mô hình* là công bằng.

Bảng 3.6: Re-ranking BM25 top-100 trên MS MARCO. Pipeline SSR được kiểm tra lại: mã thưa của query trùng 235/235 dòng với hàm của repo, điểm khớp kernel exact tới $1{,}4 \cdot 10^{-6}$.

Hệ thống	dev-small MRR@10	dev-small nDCG@10	TREC DL19 nDCG@10
BM25 (không re-rank)	18,38	22,83	50,72
ColBERTv2	**35,86**	**40,92**	**72,72**
SSR-tok (checkpoint công bố)	23,18	28,29	57,96
Backbone dense của SSR (không SAE)	15,94	20,34	47,43

Ngay trên dữ liệu in-domain, checkpoint công bố kém ColBERTv2 12,7 điểm MRR@10 và 14,8 điểm nDCG@10 trên DL19, cùng chiều với các split sạch ở mục 3.5 và trái với Bảng 1 (SSR-tok 45,2 so với ColBERTv2 39,8). Tầng SAE vẫn có ích so với chính backbone của nó (+7,2 MRR@10), nhưng backbone này là một retriever dense yếu, thấp hơn cả BM25, cho thấy BERT đã bị thay đổi nhiều trong quá trình huấn luyện (tác giả xác nhận không đóng băng BERT).

3.7. So sánh hiệu năng với ColBERTv2 + PLAID

Các tuyên bố về hiệu năng được kiểm tra trên FiQA (57.638 tài liệu, khoảng 9,8 triệu token) với PLAID của PyLate/fast-plaid (nén 2 bit, chạy GPU; bản CPU không dựng xong chỉ mục sau 50 phút nên không đưa vào). Độ trễ là một query mỗi lần, top-100, sau khởi động.

Bảng 3.7: Hiệu năng trên FiQA. Thời gian mã hóa corpus không tính vào "dựng chỉ mục" (ColBERTv2 ≈260 s, SSR ≈420 s trên GPU).

Hệ thống	Dựng index	Kích thước	ms/query (p95)	Thiết bị	nDCG test	nDCG dev
ColBERTv2 + PLAID	32,6 s	0,59 GiB	44,7 (51)	GPU	35,73	34,93
SSR-tok, repo đã sửa (exact)	164 s	2,41 GiB	1.635	CPU 1 luồng	41,41	22,80
SSR-tok, `fast_hyb` (exact, nhóm)	2,9 s	3,96 GiB	59,0 (93)	CPU 12 luồng	41,41	22,80
SSR-tok, LB-rerank t=8/1000 (nhóm)	2,9 s	3,96 GiB	36,7 (57)	CPU 12 luồng	41,41	22,81
SSR-tok, exact GPU (nhóm)	không có	3,2 GB VRAM	55,9 (60)	GPU	41,41	22,80
SSR++ cấu hình bài (bản tăng tốc)	2,9 s	3,96 GiB	11,7 (17)	CPU 12 luồng	36,86	18,11

Listing 3.3: Trích log đo PLAID (`logs/plaid_bench.log`).

```
[plaid] index cuda: build 32.6s, 603 MiB, encode 284s
[plaid] test cuda: {'ndcg@10': 35.727, 'mean_ms': 44.70, 'p50_ms': 44.44, 'p95_ms': 51.15, 'n_queries': 648}
[plaid] dev cuda:  {'ndcg@10': 34.934, 'mean_ms': 44.18, 'p50_ms': 44.07, 'p95_ms': 50.74, 'n_queries': 500}
```

Về chất lượng trên cùng FiQA: trên split dev (sạch) SSR-tok đạt 22,80 so với 34,93 của PLAID; SSR chỉ vượt PLAID trên split test. Nguyên nhân SSR mã hóa chậm hơn ColBERTv2 là tài liệu dài tới 512 token (ColBERTv2: 300) và tầng SAE sinh hàng 16.384 chiều cho mỗi token; với batch 32 các tensor này tràn 8 GB VRAM và chậm đi khoảng 8 lần, nên nhóm mã hóa với batch 16.

Bảng 3.8: Đối chiếu từng tuyên bố về hiệu năng của bài báo với số đo.

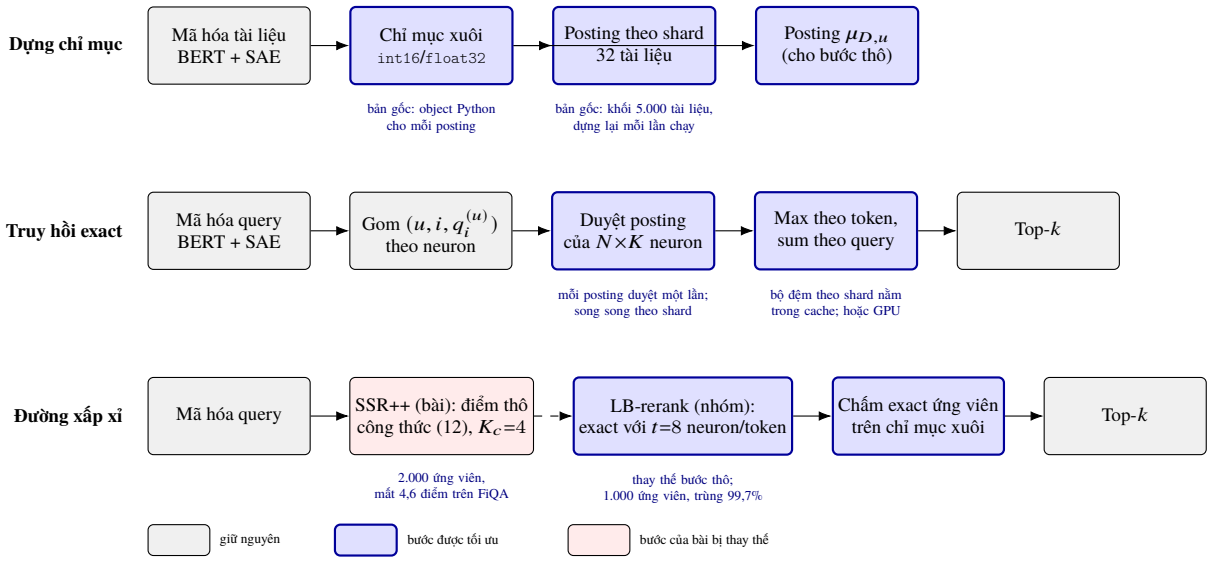
Bài báo nói	Đo được (FiQA)	Đánh giá
Dựng chỉ mục nhanh hơn 15× vì bỏ K-means (7,3 h so với 122,9 h trên MS MARCO)	K-means + nén của PLAID chỉ mất 32,6 s trên GPU; phần tốn thời gian là mã hóa. Tổng: PLAID ≈290 s, SSR ≈420 s	Không thấy ở quy mô này
PLAID phải dựng lại khi thêm tài liệu, SSR chỉ cần thêm posting	fast-plaid có sẵn `add_documents`, `remove_documents`, `update_documents`	Phóng đại
Truy hồi nhanh gần 2× PLAID	SSR exact 56–59 ms so với PLAID 44,7 ms; chỉ SSR++ nhanh hơn rõ (11,7 ms) nhưng mất 4,6 điểm	Đúng khi chấp nhận mất chất lượng
Chỉ mục nhỏ hơn PLAID (18,5 GB so với 22,1 GB)	SSR 2,4–4,0 GiB so với PLAID 0,59 GiB, lớn hơn 4–7 lần	Ngược chiều
SSR++ hầu như không mất chất lượng (Bảng 5)	FiQA test 41,41 → 36,86; dev 22,80 → 18,11	Không đúng trên corpus lớn
$h = 16.384$ neuron (Bảng 6)	Checkpoint chỉ có 8.192 neuron hoạt động; nửa còn lại trọng số 0, bias -10	Khác cấu hình

3.8. Tối ưu thuật toán truy hồi

Mục tiêu là giảm độ trễ của phép truy hồi exact (công thức (2.4)) mà giữ nguyên kết quả; mọi so sánh chất lượng của phương án xấp xỉ dùng split sạch. Mốc so sánh là bản exact đã sửa của repo (một luồng, corpus chia khối 5.000 tài liệu).

3.8.1. Vị trí tối ưu trong pipeline truy hồi

Hình 3.2 mô tả pipeline xử lý một query của SSR sau khi sửa và đánh dấu các bước được tối ưu. Phần mã hóa query (BERT + SAE) và phần chọn top-k giữ nguyên; nhóm thay đổi ba chỗ: cách dựng và lưu chỉ mục, hai bước chấm điểm trên chỉ mục (duyệt posting và gộp max/sum), và bước sinh ứng viên của đường xấp xỉ. Bảng 3.9 liệt kê từng bước với cách làm của bản gốc và của bản tối ưu.



Hình 3.2: Pipeline truy hồi của SSR và vị trí các tối ưu của nhóm. Hàng trên: dựng chỉ mục; hàng giữa: truy hồi exact (công thức (4)); hàng dưới: đường xấp xỉ, trong đó bước thô của SSR++ được thay bằng LB-rerank.

Tóm lại, phần tối ưu không đụng tới mô hình hay cách biểu diễn: biểu diễn thưa và công thức chấm điểm giữ nguyên, nên mọi kết quả xếp hạng của đường exact trùng khớp với bản gốc. Các thay đổi nằm ở cấu trúc dữ liệu của chỉ mục và thứ tự thực hiện các vòng lặp trong hai bước chấm điểm, cộng thêm một cách chọn ứng viên khác cho đường xấp xỉ. Các mục sau trình bày chi tiết từng phần.

Bảng 3.9: Từng bước của pipeline: cách làm của bản gốc (repo đã sửa) và của bản tối ưu.

Bước	Bản gốc (repo đã sửa)	Bản tối ưu	Kết quả (FiQA)
Dựng chỉ mục	Mỗi posting là một object Python, sắp xếp bằng Python; corpus chia khối 5.000 tài liệu và dựng lại ở mỗi lần chạy	Chỉ mục xuôi `fw_lat int16[n_tok,K]`, `fw_val float32[n_tok,K]`; posting đếm và điền bằng numba, lưu shard-major (khóa = shard×h + neuron)	4,8 s thay vì ≈280 s; RAM đỉnh 7,5 GB thay vì 24,5 GB
Mã hóa query, gom theo neuron	Giữ nguyên	Giữ nguyên	–
Duyệt posting	Với mỗi token query và mỗi neuron của nó, duyệt lại posting của neuron đó (một neuron chung cho m token bị duyệt m lần); một luồng	Mỗi neuron của query duyệt một lần, cập nhật mọi token chứa nó (cập nhật dạng vector khi ≥ 3 token, vô hướng khi ít hơn); các shard chạy song song (`prange`)	Chiếm phần lớn mức tăng tốc 5× (1 luồng) đến 30× (12 luồng)
Max theo token, sum theo query	Bộ đệm $N \times 5000$ tài liệu, không nằm trong cache; gộp sau khi duyệt xong cả khối	Gộp ngay trong shard trên bộ đệm $n_{\text{tok}} \times N$ cỡ vài KB; bản GPU thay hai bước này bằng SpMM (CSR) + `segment_reduce`	GPU: 56 ms/query
Top-k	`argpartition`	Giữ nguyên	–
Sinh ứng viên (đường xấp xỉ)	Điểm thô theo công thức (12) trên posting μ, $K_{\text{coarse}} = 4$, 2.000 ứng viên	LB-rerank: chấm exact với query cắt còn $t = 8$ neuron mỗi token (cận dưới của công thức (4)), 1.000 ứng viên, rồi chấm exact trên chỉ mục xuôi	nDCG 22,81 so với 18,11 của SSR++; 0,67 thời gian exact

3.8.2. Kernel exact mới `fast_hyb`

Phân tích bản gốc cho thấy ba điểm tốn thời gian: (i) bộ đệm điểm theo (token, tài liệu) có kích thước $N \times 5000$ không nằm trong cache; (ii) mỗi neuron của query được duyệt lại một lần cho mỗi token query chứa nó; (iii) chỉ chạy một luồng. Kernel mới (`repro/opt/ssr_opt/fast_exact.py`) thay đổi như sau:

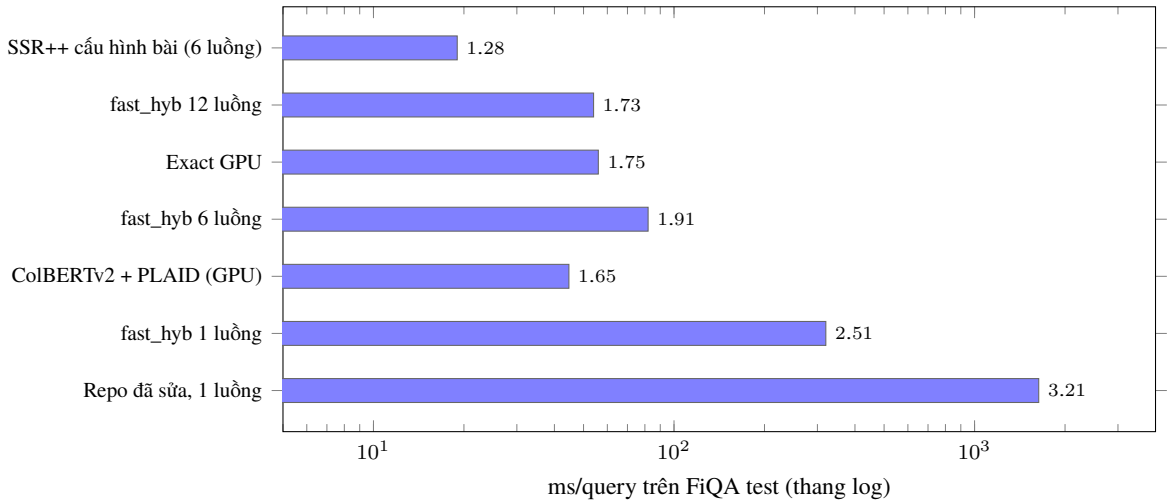
- **Chia nhỏ theo tài liệu:** corpus chia thành các shard 32 tài liệu liên tiếp; posting được lưu shard-major (khóa = shard ×h + neuron) nên bộ đệm nháp của một shard chỉ vài KB và nằm gọn trong cache L1/L2. Các shard xử lý song song bằng numba `prange`.
- **Duyệt mỗi posting một lần:** trong một shard, với mỗi neuron u của query, duyệt posting (d, t, v) của u đúng một lần và cập nhật *mọi* token query chứa u (bản gốc duyệt lại cho từng token).
- **Chỉ mục xuôi (forward index):** lưu thêm `fw_lat int16[n_tok, K]` và `fw_val float32[n_tok, K]` để chấm chính xác một tập tài liệu bất kỳ (dùng cho bước tinh của SSR++ và LB-rerank). Chỉ mục FiQA dựng trong 4,8 s với đỉnh RAM 7,5 GB, so với 174 s + 107 s và 24,5 GB của bản gốc.
- **Giữ thứ tự cộng:** các tích được cộng theo thứ tự neuron tăng dần như kernel gốc, nên điểm số giống hệt từng bit; test `tests/test_fast_exact.py` kiểm tra chênh lệch bằng 0 trên mọi tài liệu và mọi query của NFCorpus và SciFact.

Bảng 3.10: Độ trễ của các kernel exact (kết quả xếp hạng giống hệt bản gốc).

Tập	Phương pháp	Luồng	nDCG@10	ms/query (p95)	Tăng tốc
FiQA test	Repo đã sửa (mặc định)	1	41,41	1.635	1×
FiQA test	`fast_hyb`	1	41,41	320 (497)	5,1×
FiQA test	`fast_hyb`	6	41,41	82 (127)	19,9×
FiQA test	`fast_hyb`	12	41,41	54 (82)	30,0×
FiQA test	Exact trên GPU (CSR + segment max)	GPU	41,41	56 (60)	29,3×
FiQA test	SSR++ cấu hình bài (bản tăng tốc)	6	36,86	19 (27)	86×
SciFact test	Repo đã sửa → `fast_hyb`	1 → 12	87,83	354 → 12,8	28×
NFCorpus test	Repo đã sửa → `fast_hyb`	1 → 12	43,99	43,6 → 2,5	17×

3.8.3. Kernel exact trên GPU

Nhóm cài thêm một bản exact trên GPU: chỉ mục lưu dạng CSR (chỉ số int64), tích giữa query và token tính bằng SpMM rồi lấy max theo tài liệu bằng `torch.segment_reduce`. Hai điểm cần chú ý khi chạy trên Windows: phải mã hóa toàn bộ query trước rồi giải phóng mô hình để tránh tràn VRAM sang bộ nhớ chia sẻ (chậm 8 lần), và



Hình 3.3: Độ trễ truy vấn trên FiQA. Mọi phương án của SSR trừ SSR++ đều cho kết quả xếp hạng giống hệt bản gốc.

`segment_reduce` nhanh hơn nhiều so với `scatter_reduce` với tensor mở rộng. Kết quả 56 ms/query, ngang bản CPU 12 luồng.

3.8.4. Phương án xấp xỉ mới: LB-rerank

SSR++ lọc thô bằng (2.8), tức bỏ $K - K_{\text{coarse}} = 28$ neuron của mỗi token query, nên trên FiQA mất nhiều ứng viên. Nhóm đề xuất *LB-rerank*: thay vì điểm thô trên posting μ, xếp ứng viên bằng MaxSim *chính xác* của phiên bản query chỉ giữ t neuron lớn nhất của mỗi token. Vì mọi giá trị không âm, điểm này là một *cận dưới* của (2.4) nhưng vẫn ràng buộc các neuron thuộc cùng một token tài liệu, nên xếp hạng gần với điểm thật hơn nhiều. Sau đó chấm exact (đủ K) trên 1.000 ứng viên bằng chỉ mục xuôi.

Bảng 3.11: Các phương án xấp xỉ trên FiQA dev (6 luồng; exact = 82 ms, nDCG@10 22,80).

Phương án	nDCG@10	Recall@100	Trùng top-10 với exact	Thời gian / exact
SSR++ cấu hình bài (K_{coarse} 4, 2.000 ứng viên)	18,11	33,10	0,59	0,22
SSR++ K_{coarse} 4, 10.000 ứng viên	22,25	48,74	0,88	0,73
SSR++ K_{coarse} 32, 2.000 ứng viên	16,30	29,83	0,51	0,37
LB-rerank $t = 8$, 1.000 ứng viên	**22,81**	**50,72**	**0,997**	0,67
LB-rerank $t = 4$, 1.000 ứng viên	22,40	46,23	0,91	0,52

LB-rerank $t = 8$ gần như không mất điểm (top-10 trùng 99,7% với exact) ở 0,67 lần thời gian; FiQA train (1.000 query) cho cùng xu hướng (exact 20,91; LB-rerank 20,91; SSR++ cấu hình bài 17,90).

3.8.5. Các hướng đã thử nhưng không hiệu quả

- **Dừng sớm an toàn bằng cận trên** (công thức (12) với đủ K neuron, kết hợp chặn Cauchy-Schwarz theo chuẩn token): đúng về toán và cho top-k giống hệt, nhưng cận quá lỏng: 97,6% tài liệu NFCorpus vẫn nằm trên ngưỡng top-100, nên chậm hơn quét toàn bộ 2–4 lần. Lý do là tích vô hướng giữa các token SAE rất nhỏ (ngưỡng ≈0,22–0,32 mỗi token query) trong khi độ lỏng của cận là 0,34–0,51.
- **Cắt tỉa theo khối (block-max)** như bài mô tả: cận theo khối còn lỏng hơn cận theo tài liệu nên không triển khai tiếp. Vì vậy nhóm cho rằng cơ chế "bỏ qua khối theo UB" của bài đóng góp ít vào tốc độ.
- **Gộp nhiều query một lần:** tăng thông lượng tối đa khoảng 1,2×.

3.9. Thông tin từ các issue và các phiên bản của bài báo

3.9.1. Trả lời của tác giả trên GitHub

Hai nhóm độc lập (issue #1, 07/2026) huấn luyện lại theo repo chỉ đạt NFCorpus nDCG@10 0,017–0,024, và 0,108 sau khi tự sửa lỗi nạp SAE (so với 0,388 trong bài). Tác giả trả lời qua tài khoản tạm, xác nhận ba lỗi (nạp SAE ngẫu nhiên, 1 thay vì 32 hard negative, thiếu số hạng $4k$) và hứa đẩy bản sửa, nhưng repo không có commit mới. Bảng 3.12 đối chiếu các thông tin bài không ghi. Lỗi công thức chấm điểm (mục A.2) chưa được ai trong issue phát

Bảng 3.12: Thông tin từ trả lời của tác giả và đối chiếu của nhóm.

Tác giả nói	Đối chiếu
Token `[MASK]` mở rộng query bị loại trước MaxSim; nhiệt độ $\tau = 0{,}3$	Khớp config checkpoint (`do_query_expansion: false`) và cách chạy của nhóm
BERT không đóng băng; lr 10^{-3} trong Bảng 6 "không ổn định", khoảng 10^{-6} tốt hơn	Bảng 6 ghi 10^{-3}; nhóm thêm preset `train_issue1_setup.sh`
32 hard negative mỗi query, 2 query × 32 triplet mỗi batch 64; warmup 20.000 bước ≈4% của 2 epoch	Nhóm cài đặt (`--negatives-per-query`, `ssr/samplers.py`), chạy thử được với batch 16
Số hạng $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$ có dùng khi huấn luyện, chỉ sót khi công bố	Nhóm thêm lại
Trên NFCorpus hơn 70% neuron được kích hoạt (đo bằng script thăm dò)	Checkpoint công bố: 7.453/16.384 neuron (45,5%); không thể vượt 50% vì chỉ 8.192 neuron hoạt động

hiện; vì lệnh đánh giá mặc định của repo đi qua đúng đường này, con số 0,108 của họ nhiều khả năng cũng bị kéo xuống bởi lỗi đó.

3.9.2. Các phiên bản của bài báo

- **arXiv v3 (03/06/2026) và bản PMLR:** Bảng 1 ghi "MS MARCO báo cáo bằng MRR@10"; số của SSR-tok (45,2) thực ra là nDCG@10, còn ColBERTv2 (39,8), PLAID (39,7) là MRR@10. Hai độ đo này không so được với nhau (mục 2.7).
- **arXiv v4 (28/09/2026):** đổi MS MARCO sang nDCG@10 ở mọi bảng và cập nhật baseline: ColBERTv2 39,8 → 44,2; PLAID 39,7 → 44,1; COIL 35,3 → 40,9; ColBERT 36,0 → 41,1. Lợi thế in-domain của SSR giảm từ +5,4 xuống +1,0. V4 cũng sửa trung bình của ColBERT (41,5 → 44,8) và dòng XTR ở Bảng 9 (47,6 → 45,0), đúng hai mâu thuẫn tính lại được từ v3; thí nghiệm theo quy mô dữ liệu đổi từ "SSR giảm 5,3% so với 8,9%" thành "4,2% so với 4,7%".
- **Bản chính thức PMLR** đổi nhãn sang nDCG@10 nhưng giữ số baseline của v3, nên so sánh lệch chỉ số vẫn nằm trong bản đã xuất bản. Bảng 15 ở cả hai bản vẫn ghi MRR@10: SSR-tok 39,5, PLAID 39,6, EMVB 39,8, tức theo chính bảng này SSR chỉ ngang PLAID trên MS MARCO.

3.10. Giới hạn của thực nghiệm

- MS MARCO chỉ kiểm bằng re-ranking; full retrieval cần ≥70 GB RAM cho chỉ mục mức token.
- Chưa huấn luyện lại đầy đủ (khoảng 500k bước GPU) và chưa kiểm biến thể Llama-Embed-8B (cần hơn 8 GB VRAM).
- Độ trễ đo trên RTX 5060 / i5-12400F, không so trực tiếp được với A100 trong bài; chỉ so sánh tương đối giữa các hệ thống trên cùng máy.
- Mọi kết luận về chất lượng áp dụng cho checkpoint công bố trên HuggingFace, không phải mô hình chưa công bố đã dùng cho Bảng 1.

CHƯƠNG 4. KẾT LUẬN

4.1. Điểm mạnh của phương pháp SSR

- **Ý tưởng đơn giản, tận dụng cấu trúc dữ liệu sẵn có:** biểu diễn thưa TopK biến mỗi neuron thành một "từ", cho phép dùng chỉ mục ngược, cấu trúc đã được tối ưu nhiều năm cho tìm kiếm từ khóa, mà vẫn giữ tương tác giữa các token của ColBERT. Không cần K-means, codebook hay giải nén residual.
- **Chỉ mục cập nhật được:** thêm tài liệu chỉ là thêm posting, không phụ thuộc vào codebook đã học.
- **Tiềm năng tốc độ:** với kernel phù hợp (mục 3.8), truy hồi exact đạt 54–59 ms/query trên CPU cho 57k tài liệu và dựng chỉ mục trong vài giây; biểu diễn thưa cũng cho phép xấp xỉ có kiểm soát (LB-rerank) gần như không mất chất lượng.
- **Có phân tích lý thuyết:** Phụ lục A cho chặn sai số giữa MaxSim thưa và MaxSim dense.

4.2. Hạn chế của bài báo và mô hình

- **Không tái lập được từ mã nguồn công bố:** mã chấm điểm sai công thức (4), hàm nạp checkpoint tạo SAE ngẫu nhiên, thiếu số hạng tổn thất, bộ nạp dữ liệu khác mô tả của tác giả. Chính tác giả xác nhận mã công bố không phải mã tạo ra số trong bài và cách huấn luyện thật (lr, số negative, nhiệt độ) khác Bảng 6.
- **Checkpoint công bố có dấu hiệu tiếp xúc dữ liệu test:** điểm chỉ vọt lên ở split test của BEIR, trong khi trên split sạch và trên MS MARCO in-domain nó kém ColBERTv2 từ 0,7 đến 12,7 điểm. Checkpoint cũng chỉ dùng 8.192/16.384 neuron, khác cấu hình trong bài.
- **So sánh lệch chỉ số trên MS MARCO** ở bản v3 và bản PMLR; sau khi v4 sửa, lợi thế in-domain chỉ còn +1,0.
- **Các tuyên bố hiệu năng chỉ đúng một phần:** ở quy mô 57k tài liệu, thời gian dựng chỉ mục bị chi phối bởi mã hóa (SAE làm chậm hơn ColBERTv2), chỉ mục lớn hơn PLAID 4–7 lần, và SSR++ mất 4,6 điểm trên corpus lớn.
- **Chi phí bộ nhớ khi mã hóa:** tầng SAE sinh hàng 16.384 chiều cho mỗi token, dễ tràn VRAM với batch lớn hoặc tài liệu dài.

4.3. Khả năng ứng dụng thực tiễn

- **Tìm kiếm ngữ nghĩa trên hạ tầng từ khóa:** vì chỉ mục của SSR là chỉ mục ngược thông thường, có thể nhúng vào các máy tìm kiếm sẵn có (Lucene/Elasticsearch) dưới dạng "từ giả", tương tự cách SPLADE được triển khai, mà vẫn giữ tương tác đa vector.
- **Kho tài liệu thay đổi liên tục:** tin tức, hỗ trợ khách hàng, tài liệu nội bộ, nơi việc dựng lại codebook K-means theo chu kỳ là gánh nặng.
- **Hệ thống RAG:** bước truy hồi cho mô hình ngôn ngữ lớn cần chất lượng tốt hơn BM25 nhưng độ trễ thấp và chi phí dự đoán được; kết hợp với kernel `fast_hyb` hoặc LB-rerank của nhóm, SSR đạt vài chục ms/query trên CPU.
- **Điều kiện áp dụng:** cần một checkpoint được huấn luyện minh bạch; với checkpoint hiện tại, chất lượng trên dữ liệu sạch chưa bằng ColBERTv2.

4.4. Hướng phát triển tiếp theo

- **Huấn luyện lại minh bạch:** dùng mã đã sửa (đủ số hạng $4k$, 32 negative gom theo query, $\tau = 0{,}3$) trên GPU lớn hơn để tạo checkpoint có thể kiểm chứng; đánh giá trên split sạch và full retrieval MS MARCO.
- **Giảm kích thước chỉ mục:** lượng tử hóa giá trị posting (8 bit), nén chỉ số token, hoặc giảm độ dài tài liệu về 300 token như ColBERTv2 để thu hẹp khoảng cách 4–7 lần với PLAID.
- **Cắt tỉa an toàn hiệu quả hơn:** cận trên hiện quá lỏng; có thể học cận theo cặp neuron hoặc dùng cấu trúc "impact-ordered" kết hợp với LB-rerank để có dừng sớm có bảo đảm.
- **Tích hợp tín hiệu toàn cục:** kết hợp SSR-CLS (SAE trên token [CLS]) hoặc trộn với BM25 để bù cho token thưa trên tài liệu dài.
- **Mở rộng backbone:** kiểm chứng biến thể Llama-Embed-8B của bài trên phần cứng đủ VRAM, và thử SAE trên các mô hình embedding đa ngôn ngữ cho tiếng Việt.

KẾT LUẬN

Báo cáo đã phân tích bài báo *No More K-means* và phương pháp SSR: dùng bộ tự mã hóa thưa TopK để đưa truy hồi đa vector về chỉ mục ngược, bỏ bước K-means của PLAID. Về lý thuyết, đây là hướng đi hợp lý, kết hợp được ưu điểm của truy hồi từ vựng và truy hồi đa vector.

Về thực nghiệm, nhóm đã chạy lại mã nguồn và checkpoint công bố, phát hiện và sửa 9 nhóm lỗi (trong đó lỗi công thức chấm điểm trên chỉ mục ngược chưa từng được báo cáo), đóng gói thành patch có test, và kiểm chứng kết quả bằng nhiều cách tính độc lập cùng baseline đối chứng. Kết quả cho thấy mã công bố không tái lập được số trong bài; checkpoint công bố chỉ vượt ColBERTv2 trên đúng split test của BEIR và kém ColBERTv2 trên split sạch cũng như trên MS MARCO in-domain; một số tuyên bố hiệu năng không đúng ở quy mô nhóm đo được; và bản arXiv v4 đã tự sửa một lỗi so sánh lệch chỉ số.

Về tối ưu, nhóm thiết kế lại kernel chấm điểm exact (nhanh hơn 19–30 lần trên FiQA, kết quả giống hệt từng bit, dựng chỉ mục 4,8 s thay vì khoảng 280 s), cài bản GPU, và đề xuất phép xấp xỉ LB-rerank gần như không mất chất lượng, tốt hơn SSR++ của bài trên corpus lớn.

Qua đề tài, nhóm rút ra rằng kết quả công bố cần được kiểm chứng bằng baseline đối chứng trên cùng pipeline, bằng split sạch bên cạnh split test, và bằng cách đối chiếu mã nguồn với từng công thức trong bài. Đây là kinh nghiệm chính nhóm thu được từ môn học, bên cạnh kiến thức về chỉ mục ngược, MaxSim và biểu diễn thưa.

TÀI LIỆU THAM KHẢO

[1] L. Guo, Y. Wang, T. Wen, A. Feng, S. Jegelka, C. You, "No More K-means: Single-Stage Sparse Coding for Efficient Multi-Vector Retrieval," *Proc. 43rd International Conference on Machine Learning (ICML)*, PMLR 306, 2026. arXiv:2605.30120 (v3 03/06/2026; v4 28/09/2026).

[2] O. Khattab, M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," *Proc. SIGIR*, 2020, pp. 39–48.

[3] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, M. Zaharia, "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction," *Proc. NAACL*, 2022, pp. 3715–3734.

[4] K. Santhanam, O. Khattab, C. Potts, M. Zaharia, "PLAID: An Efficient Engine for Late Interaction Retrieval," *Proc. CIKM*, 2022, pp. 1747–1756.

[5] F. M. Nardini, C. Rulli, R. Venturini, "Efficient Multi-Vector Dense Retrieval with Bit Vectors," *Proc. ECIR*, 2024.

[6] T. Formal, B. Piwowarski, S. Clinchant, "SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking," *Proc. SIGIR*, 2021; C. Lassance, H. Déjean, T. Formal, S. Clinchant, "SPLADE-v3: New Baselines for SPLADE," arXiv:2403.06789, 2024.

[7] L. Gao, T. Dupré la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, J. Wu, "Scaling and Evaluating Sparse Autoencoders," arXiv:2406.04093, 2024 (ICLR 2025).

[8] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, I. Gurevych, "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models," *NeurIPS Datasets and Benchmarks*, 2021.

[9] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, L. Deng, "MS MARCO: A Human Generated MAchine Reading COmprehension Dataset," *CoCo@NeurIPS*, 2016.

[10] S. E. Robertson, S. Walker, S. Jones, M. Hancock-Beaulieu, M. Gatford, "Okapi at TREC-3," *Proc. TREC-3*, 1995.

[11] K. Järvelin, J. Kekäläinen, "Cumulated Gain-based Evaluation of IR Techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.

[12] P. Yang, H. Fang, J. Lin, "Anserini: Enabling the Use of Lucene for Information Retrieval Research," *Proc. SIGIR*, 2017, pp. 1253–1256.

[13] A. Chaffin, R. Sourty, "PyLate: Flexible Training and Retrieval for Late Interaction Models," LightOn, 2024. https://github.com/lightonai/pylate.

[14] Mã nguồn chính thức của SSR: https://github.com/Y-Research-SBU/SSR (commit c64bc05, 26/06/2026); Issue #1 "Difficulty reproducing BEIR numbers from Table 1" (24/07/2026) và Issue #2 "Is the 4k reconstruction loss missing from the training objective?" (04/08/2026).

[15] Checkpoint SSR-tok: https://huggingface.co/Y-Research-Group/SSR (đưa lên 05/10/2026).

[16] Bản chính thức trên PMLR: https://proceedings.mlr.press/v306/guo26x.html.

[17] Mã nguồn, patch, script đánh giá và kết quả của nhóm: https://github.com/duykhanhxx03/ssr-repro-and-optimization.

PHỤ LỤC A. CHI TIẾT CÁC LỖI TRONG MÃ NGUỒN VÀ BẢN SỬA

Phụ lục này trình bày chi tiết các lỗi đã tóm tắt trong Bảng 3.3 (mục 3.3), kèm đoạn mã trước và sau khi sửa. Toàn bộ bản sửa nằm trong patch `ssr_paper_fixes.patch` của repo nhóm, áp dụng được lên commit `c64bc05` của mã nguồn gốc.

A.1. Lỗi 2: nạp checkpoint tạo SAE ngẫu nhiên

Hàm `load_ssr` gọi `SSR(model_name_or_path=...)`; vì PyLate chỉ giữ các module `Transformer`/`Dense` khi nạp, module `SparseAutoencoder` bị bỏ và một SAE mới với $4 \times 768 = 3072$ latent được khởi tạo ngẫu nhiên. Mọi đánh giá bằng lệnh mặc định của repo do đó chạy trên một SAE chưa huấn luyện. Một người dùng trong issue #1 cũng phát hiện lỗi này độc lập.

Listing A.1: `ssr/retrieval/eval_mteb.py`: trước (trên) và sau khi sửa (dưới).

```python
def load_ssr(model_path: Path, device: str) -> SSR:
    model = SSR(model_name_or_path=str(model_path), device=device)   # SAE ngau nhien
    model.eval()
    return model

def load_ssr(model_path: Path, device: str, token_scope=None) -> SSR:
    # SSR(model_name_or_path=...) would attach a randomly initialised SAE.
    return load_ssr_checkpoint(str(model_path), device=device,
                               token_scope=token_scope)
```

Hàm mới `load_ssr_checkpoint` (trong `ssr/model.py`) đọc `modules.json`, nạp SAE từ thư mục con `1_SparseAutoencoder` (lọc các khóa config lạ), và khôi phục các thiết lập ColBERT lưu trong `config_sentence_transformers.json`: tiền tố `[Q]`/`[D]`, độ dài query 192 và tài liệu 512, `do_query_expansion = false`, danh sách skiplist.

A.2. Lỗi 1: công thức chấm điểm trên chỉ mục ngược

Lỗi này ảnh hưởng lớn nhất đến kết quả và chưa được nêu trong các issue. Bộ cộng dồn của chế độ `exact` làm việc theo từng neuron: với mỗi neuron u của token query i, nó lấy giá trị lớn nhất của u trong mỗi tài liệu rồi cập nhật ô (i, D) bằng phép `maximum`:

Listing A.2: `inverted_index.py` gốc: bộ cộng dồn lấy max theo từng neuron.

```python
for q_entries, s, e in segments:          # moi segment = mot neuron u
    docs = doc_idx[s:e]; vals = values[s:e]
    out_docs, max_vals = segment_max_by_doc(docs, vals)   # mu_{D,u}
    for qt, v_q in q_entries:             # cac token query chua u
        scaled = max_vals * np.float32(v_q)
        row = local[qt]
        row[out_docs] = np.maximum(row[out_docs], scaled)  # max, khong phai cong
```

Kết quả là điểm của cặp (token query i, tài liệu D) bằng $\max_u q_i^{(u)} \mu_{D,u}$: mỗi neuron được phép chọn một token tài liệu khác nhau, và chỉ neuron lớn nhất được giữ. So với (2.4), phép tính này vừa đánh mất phép cộng trên các neuron chung, vừa không ràng buộc các neuron phải thuộc cùng một token tài liệu. Kết quả đo được: NFCorpus 8,33 và SciFact 7,46 nDCG@10, thấp hơn cả BM25.

Bản sửa (`ssr/retrieval/exact_index_maxsim.py`, kernel numba) cộng dồn theo đúng hàng (tài liệu, token tài liệu), rồi mới lấy max theo token trong tài liệu và cộng theo token query:

Listing A.3: Kernel exact mới (rút gọn): cộng theo (doc, token) rồi mới max theo token.

```
for qt in range(n_qt):                      # moi token query
    gen_state[0] += 1; gen = gen_state[0]; n_rows = 0
    for k in range(qt_offsets[qt], qt_offsets[qt + 1]):   # moi neuron u cua qt
        li = seg_rows[k]; vq = seg_vq[k]
        for p in range(offsets[li], offsets[li + 1]):     # posting (doc, token, val)
            d = doc_idx[p]
            r = row_start[d] + token_idx[p]               # hang = (doc, token)
            if row_gen[r] != gen:                         # lan dau cham hang nay
                row_gen[r] = gen; acc[r] = 0.0
                touched_rows[n_rows] = r; n_rows += 1
            acc[r] += vq * values[p]                      # SUM tren neuron chung
    for i in range(n_rows):                               # MAX theo token trong doc
        r = touched_rows[i]; d = row_doc[r]
        if doc_gen[d] != gen:
            doc_gen[d] = gen; doc_best[d] = acc[r]
            touched_docs[n_docs] = d; n_docs += 1
        elif acc[r] > doc_best[d]:
            doc_best[d] = acc[r]
    for i in range(n_docs):                               # SUM theo token query
        d = touched_docs[i]; doc_total[d] += doc_best[d]
```

Vì mọi giá trị sau TopK+ReLU đều không âm, các token tài liệu không chung neuron nào với token query đóng góp 0 và có thể bỏ qua mà không đổi kết quả max. Bản sửa được kiểm chứng bằng ba cách độc lập: (i) so với MaxSim tính trên vector thưa dạng dense bằng PyTorch (`reference_maxsim_dense`); (ii) so với một cài đặt bằng `scipy.sparse` viết riêng; (iii) so với bản "unblocked" của repo đã sửa cùng lỗi. Cả ba trùng nhau tới 10^{-6} trên mọi cặp query và tài liệu của NFCorpus và SciFact.

A.3. Lỗi 3 và 4: ánh xạ neuron và bước thô SSR++

Chỉ mục nén các neuron thực sự xuất hiện trong corpus thành chỉ số liên tiếp (`latent_remap`); đường `e2e-index` quên áp ánh xạ này lên query nên phần giao giữa neuron query và neuron tài liệu chỉ còn khoảng 2% (nDCG@10 0,012). Sau khi ánh xạ query bằng `remap_sparse_embeddings`, kết quả là 0,440, trùng với đường chuẩn.

Bước thô của SSR++ trong repo dùng một cơ chế cắt tỉa theo khối khác với (2.8) và tham số mặc định ($K_{\text{coarse}} = 8$, 32.768 ứng viên, khối 512) không khớp Phụ lục D.1. Nhóm cài đặt lại đúng (2.8) trên posting $\mu_{D,u}$ (được dựng từ posting token bằng một phép sắp xếp), lấy top-2.000 toàn cục trên mọi khối, rồi chấm tinh bằng kernel exact với mặt nạ `allowed`:

Listing A.4: Kernel lọc thô theo công thức (12).

```
@njit(cache=True)
def _coarse_totals_kernel(mu_offsets, mu_doc, mu_val, seg_rows, seg_vq, doc_total):
    for k in range(seg_rows.shape[0]):        # (neuron u, q_i^{(u)}) voi u trong top-Kcoarse
        li = seg_rows[k]; vq = seg_vq[k]
        for p in range(mu_offsets[li], mu_offsets[li + 1]):
            doc_total[mu_doc[p]] += vq * mu_val[p]     # q_i^{(u)} * mu_{D,u}
```

A.4. Lỗi 5–7 và 9: độ đo và huấn luyện

Hàm nDCG của repo đọc qrels thành tập nhị phân, làm mất mức liên quan 2 của NFCorpus và 2–3 của TREC DL; nhóm sửa để giữ `Dict[str, Dict[str, int]]` và dùng độ lợi $2^{\text{rel}} - 1$. Trong `losses.py`, `recons_4k` được tính trong forward của SAE nhưng không đưa vào tổn thất; nhóm thêm thành phần `reconstruction_4k` với hệ số 0,125 (issue #2 xác nhận tác giả có dùng số hạng này). Nhóm cũng thêm `--warmup-steps`, `--negatives-per-query` và một `QueryGroupedBatchSampler` gom các triplet của cùng query vào một batch như tác giả mô tả:

Listing A.5: `ssr/samplers.py`: gom triplet cùng query vào một batch.

```python
class QueryGroupedBatchSampler(DefaultBatchSampler):
    def __init__(self, dataset, batch_size, drop_last, ...):
        super().__init__(dataset, batch_size=batch_size, drop_last=drop_last, ...)
        queries = dataset["query"]; groups = []
        for i, q in enumerate(queries):      # cac hang lien tiep cung query = 1 nhom
            if i > 0 and q == queries[i - 1]: groups[-1].append(i)
            else: groups.append([i])
        self.groups = groups

    def _batches(self):
        g = torch.Generator(); g.manual_seed(int(self.seed) + int(self.epoch))
        order = torch.randperm(len(self.groups), generator=g).tolist()
        batches, current = [], []
        for gi in order:                     # xep nguyen nhom vao batch
            group = self.groups[gi]
            if current and len(current) + len(group) > self.batch_size:
                batches.append(current); current = []
            current = current + group
        if current: batches.append(current)
        return batches
```

Hai preset `scripts/train_table6.sh` (đúng Bảng 6) và `scripts/train_issue1_setup.sh` (cấu hình tác giả nói trong issue: lr 10^{-6}, 32 negative gom theo query, $\tau = 0{,}3$, warmup 20.000) được thêm vào. Đường huấn luyện đã chạy thử trọn vẹn trên một tập MS MARCO nhỏ (batch 16, 8 negative mỗi query; log cho thấy `reconstruction_4k` và warmup lr hoạt động, checkpoint nạp lại được). Bước này chỉ kiểm tra mã chạy được; huấn luyện đầy đủ cần khoảng $32 \times 500\text{k}/64 \approx 250\text{k}$ bước mỗi epoch với batch 64, vượt quá GPU 8 GB.